Computer Security Exam

Professors M. Carminati & S. Zanero

12/01/2022

Last (family) Name _____

First (given) Name _____

Matricola _____

Codice Persona _____

Professor: [ ] Carminati [ ] Zanero

Have you done any challenges/homework, even partially? [ ] Yes [ ] No

Do you want to withdraw? [ ] Yes [ ] No

Instructions

- **Duration**: 2 hours.
- The exam is composed of 20 pages. Check that you have all of them.
- Just as a cross-check, tell us whether you have completed homeworks by appropriately putting an "X" mark.
- The exam is an "open book". The students will be allowed to consult only the **printed course material** and **notes**. **No extra devices** (e.g., phones, iPad) are **allowed**. You will be expelled if, at any time, if you do not follow this rule.
- Shut down and store electronic devices. They will be subject to inspection if found and you may be expelled if you are found using one.
- You are not allowed to communicate with other students, and you will be expelled from the exam if you do.
- Please answer within the allowed space. Schemes are good; short answers are recommended.
- You can write in pen or pencil, any color, but avoid writing in red.
- No extra paper is allowed.
- **Always motivate your answer.**
- **If your final grade is below 12 (not considering the challenges), you will not be allowed to take the next exam call (even in the next session).**

- **DISTANCE MODE ONLY**

- You will be allowed to consult only printed documents or notes as supporting materials. No digital devices (other than the computer you are reading the exam from) will be allowed.
- On the computer, you will be allowed to open only the MS forms browser tab and the exam pdf. Any other application or document is prohibited.
- You will have to share your screen, leave your microphone opened, and your webcam must frame both you and your workspace.
- You are responsible for having all the necessary technological equipment for the correct delivery of the exam.
- If you disable the screen sharing or the audio, you will be expelled from the virtual room and your exam will be voided.
- We will ask remote students to sustain a brief oral exam to confirm the evaluation of the written exams. This could be done for all students or for a subset, at the instructor's decision.
- Regarding the submission:
  - At the end of the exam, you will be given 10 minutes only to scan your documents and prepare to upload them.
  - After 10 minutes, you will be called by name and required to click on the submit button of the MS forms under the supervision of the supervisor of the virtual room. Any submission that will not follow this procedure (e.g., you have already submitted before being called or you are not ready to submit when called) will be voided.
  - We are not responsible for any technical issue in the submission (e.g., the exam pictures are not readable, the pdf is corrupted). We will evaluate only the uploaded readable documents.
  - We will evaluate only the uploaded **readable** documents.
  - USE YOUR PERSONAL CODE (CODICE PERSONA) ONLY AS THE NAME OF THE DOCUMENT (e.g., 12345678.pdf).

PLEASE, USE THE FORMATTING SPECIFIED IN THE QUESTION

READ CAREFULLY ALL THE POINTS OF EACH QUESTION BEFORE WRITING YOUR ANSWER

Exercise 1 (9 points)

Assume that:

- The C standard library is loaded at a known address during every execution of the program, and that the address of the function `system()` is `0xf4d0e2d3`.
- Environment variables are located in the highest addresses.
- *The program is compiled for the **x86 architecture** (32 bit) and for an environment that adopts the usual **`cdecl`** calling convention. Furthermore, assume that **no compiler-level or OS-level mitigations** against the exploitation of memory corruption errors is present (unless specified otherwise).*

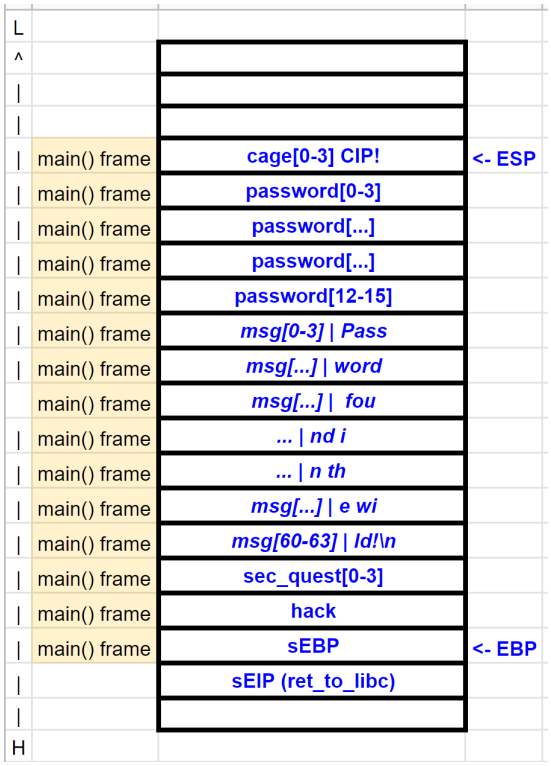
```c
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <stdint.h>

typedef struct {
   char cage[4];
   char password[16];
   char msg[64];
   char sec_quest[4];
   int hack;
} data_t;

int check_if_copromised(data_t *data){
   char msg[128];
   if (strstr(data->password, "password")){ // Check for substring
      snprintf(msg, 128, "Hacked Account! %s", data->msg);
      printf(msg);
      return 1;
      }
   return 0;
}

void reset_pass(data_t *data){
   int count=0;

   scanf("%20s", data->sec_quest); // Insert security question
   while (data->hack){
      count = count +1;
      scanf("%15s", data->password); // Insert new password
      data->hack = check_if_copromised(data);
      if (count > 3)
         abort();
   }
}

void main(int argc, char** argv) {
   data_t data;

   strcpy(data.cage, "CIP!");
   strcpy(data.msg, "Password found in the wild!\n");

   scanf("%80s", data.password); // Insert password to check
   data.hack = check_if_copromised(&data);
   reset_pass(&data);
   if (!strncmp(data.cage, "CIP!", 3) == 0 || data.hack == 1)
       abort();
}
```

[1 point] 1. The program is affected by typical buffer overflow and format string vulnerabilities. Complete the following table, focusing on a vulnerability per row.

Vulnerability	**Line**	**Modify the line to solve the detected vulnerability**
Buffer Overflow	43 **[0.25]** 27 **[0.25]**	`scanf("%48s", data.password);` **[0.25]** `scanf("%20s", data->sec_quest);` [0.25]
Format String	18 **[0.25]**	`printf(msg);` **[0.25]**

[1 point] ***2.*** Draw the stack layout after the program has executed the instruction at line 42, showing:
- Direction of growth and high-low addresses;
- The name of each allocated variable;
- The content of relevant registers (i.e., EBP, ESP);
- The functions stack frames.

Show also the content of the caller frame.



[2 points] 3. **Focus only on the stack-based buffer overflow(s) you found.** Write an exploit for this vulnerability that **must execute the following shellcode**, composed of 16 bytes, which opens a shell: **`0x20 0x30 0x40 0x50 0x60 0x70 0x80 0x90 0x20 0x30 0x40 0x50 0x60 0x70 0x80 0x90`**.

3.a) Describe **all the steps and assumptions** required for the successful exploitation of the vulnerability. Include also any assumption on how you must call the program (e.g., the values for the command-line arguments required to trigger the exploit correctly and/or environment variables, if any).

Solution Draft:

- *Exploit vuln at line 43 to insert the payload (NOP+shellcode) in password[] and msg[]*
- *Exploit the vuln at line 27 to overwrite the sEIP of the main with the address of the payload inserted in the previous step (data.hack must be != 1)*

Cage is "untouched" by the exploit

Alternative solution

- *Put the the payload (NOP+shellcode) in an env variable*
- *Exploit the vuln at line 27 to overwrite the sEIP of the main with the address of the payload inserted in the previous step (data.hack must be != 1)*

-

3.b) Make sure that you show how the exploit will appear in the process memory with respect to the stack layout right **before and after the execution of the vulnerable line** during the program exploitation.

		before line 27			*after line 27*	
L				reset_pass() frame	count	<- ESP
^				reset_pass() frame	sEBP	
|				main() frame	sEIP	
|				main() frame	*data	
|	main() frame	cage[0-3] CIP!	<- ESP	main() frame	cage[0-3] CIP!	
|	main() frame	*password[0-3] | 0x90 0x90 0x90 0x90*		main() frame	*password[0-3] | 0x90 0x90 0x90 0x90*	<- X
|	main() frame	*password[...] | 0x90 0x90 0x90 0x90*		main() frame	*password[...] | 0x90 0x90 0x90 0x90*	
|	main() frame	*password[...] | 0x20 0x30 0x40 0x50*		main() frame	*password[...] | 0x20 0x30 0x40 0x50*	
|	main() frame	*password[12-15] | 0x60 0x70 0x80 0x90*		main() frame	*password[12-15] | 0x60 0x70 0x80 0x90*	
|	main() frame	*msg[0-3] | 0x20 0x30 0x40 0x50*		main() frame	*msg[0-3] | 0x20 0x30 0x40 0x50*	
|	main() frame	*msg[...] | 0x60 0x70 0x80 0x90*		main() frame	*msg[...] | 0x60 0x70 0x80 0x90*	
	main() frame	*msg[...] | not_relevant*		main() frame	*msg[...] | not_relevant*	
|	main() frame	*... | not_relevant*		main() frame	*... | not_relevant*	
|	main() frame	*... | not_relevant*		main() frame	*... | not_relevant*	
|	main() frame	*msg[...] | not_relevant*		main() frame	*msg[...] | not_relevant*	
|	main() frame	*msg[60-63] | not_relevant*		main() frame	*msg[60-63] | not_relevant*	
|	main() frame	sec_quest[0-3]		main() frame	sec_quest[0-3] not_relevant	
|	main() frame	hack		main() frame	hack | 0	
|	main() frame	sEBP	<- EBP	main() frame	sEBP | not_relevant	
|		sEIP (ret_to_libc)			sEIP (ret_to_libc) | ~X	
|						
H						

[1 point] 4. <u>Focus only on the format string vulnerability you found</u>. Write an exploit <u>that executes the previous shellcode</u>, assuming that you have already prepared the memory with the correct arguments.

- The address of the return address (*saved EIP*) of the function exploited is **`0xffbfdd9c`** (i.e., where to write)
- The address of the first instruction of the **`shellcode`** is in an **environment variable** at **`0xbeefdead`** (i.e., what to write)
- The displacement on the stack of the vulnerable function's argument is: **4**

Knowing that *dec(0xbeef) = 48879 and dec(0x**dead**)=57005*, write the exploit clearly, detailing all the components of the format string and all the steps that lead to successful exploitation.

[1 points] Not possible to exploit with format string only.

[0.25] *Format string vulnerability, in the msg variable, which however is not received as input, but hardcoded. To exploit the format string it is necessary to write the "format string payload" exploiting the buffer overflow at line 43 in the msg variable. Also, the password must contain "password" string.*

0.5 We need to write 0xbeefdead to 0xffbfdd9c. As 0xbeef < 0xdead, we need to swap.
The general format string structure is: <where to write+2><where to write>%<low value>c%<pos>$hn%<high value>c%<pos>$hn
- Where to write = \x9c\xdd\xbf\xff
- Where to write + 2 = \x9e\xdd\xbf\xff
- we have already written 8 +16 characters
- low value = 0x49d0 -> 48879 - 24 = ...
- high value = 0xf7e3 -> 57005 - 48879 = ...
Also, the displacement on the stack of the printf's argument (i.e., the buffer message) is 4+4
Complete exploit:\x9e\xdd\xbf\xff\x9c\xdd\xbf\xff%...c$8hn%...c$9hn

[1 point] 5. Assume (for this question only) that the `clearenv()` function, which clears all environment variables from the environment table and frees the associated storage, is called at each execution of the program. Is the implemented mitigation effective to prevent **your exploit** at points 3. and 4.? If the mitigation technique is effective, explain why and describe how you would modify the exploits to bypass the mitigation. If it is not effective, please explain why. You must execute the same shellcode, no external function can be called.

3. No (Yes, if you have used env var) ...

4. Yes, it uses env var. Put the shellcode on the stack ...

[1 point] 6. Consider that the program is compiled enabling ONLY the exploit mitigation technique known as stack canary. Is it effective to prevent **your exploits** at points 3 and 4? If the mitigation technique is effective, explain why and describe how you would modify the exploit (or use a different exploitation technique) to bypass the mitigation. If it is not effective, please explain why.

3. Yes, the canary will be overwritten. Use the format string to leak the canary or use the format string directly to overwrite the sEIP. [see slides]

4. No, the canary will not be "touched". [see slides]

[1 point] 7. Consider ONLY the "Address-space layout randomization (ASLR)" mitigation technique. Is it effective to prevent **your exploits** at points 3 and 4? If the mitigation technique is effective, explain why and describe how you would modify the exploits (or use a different exploitation technique) to bypass the mitigation. If it is not effective, please explain why.

3. Yes, [see slides] ... the format string can be used to leak values of the stack.

4. Yes, [see slides] ... the format string can be used to leak values of the stack.

[1 point] 8. Consider ONLY the "Non-executable stack (NX)" mitigation technique. Is it effective to prevent **your exploits** at points 3 and 4? If the mitigation technique is effective, explain why and describe how you would modify the exploits (or use a different exploitation technique) to bypass the mitigation. If it is not effective, please explain why.

3. Yes, [see slides] + ROP or ret_to_libc

4. Yes, [see slides] + ROP or ret_to_libc

Exercise 2 (8 points)

Blue Pwnër Cult is a forum for young hackers that love progressive rock. People here can post on whatever they want, and registered users can comment on anyone's posts. You're new to this community and you really want to let people know you're worthy. Ok, maybe that's not the best way… but they also have a bug bounty program…
Come on, let's hack this website!

The relevant pseudo-code of the forum is the following:

```
01 # https://bluepwnercult.com/register
02 def register():
03     if request.method != "POST":
04           abort(403)
05     username = request.form['username']
06     password = request.form['password']
07
08     if not is_alphanum_plus(username) or not is_alphanum_plus(password):
09           abort(400)
10
11     q = "INSERT INTO User (username, password, admin) VALUES ('" + username +
"','" + password + "', False);"
12     dbquery(q)
13
14     return login_page()
15
16 # https://bluepwnercult.com/login
17 def login():
18     if request.method != "POST":
19           abort(403)
20     username = request.form['username']
21     password = request.form['password']
22
23     if not is_alphanum_plus(password):
24           abort(400)
25
26     q = "SELECT * FROM User WHERE username='" + username + "' and password='"
+ password + "'"
27     result = dbquery(q)
28     if result.empty():
29           return login_page(error="Wrong username or password.")
30
31     request.cookies['session']['username'] = username
32
33     return login_page()
34
35 # https://bluepwnercult.com/comment
36 def comment():
37     username = check_session(request.cookies['session'])
38     if request.method != "POST":
```

```
39          abort(403)
40
41    post_number = int(request.form['post_number'])
42    comment = request.form['comment']
43
44    comment = html_escape(comment)
45
46    q = "INSERT INTO Comments (post_number, author, comment) VALUES ('" +
post_number + "','" + username + "','" + comment + "');"
47    dbquery(q)
48    return
49
50 # https://bluepwnercult.com/post
51 def get_post():
52    username = check_session(request.cookies['session'])
53    if request.method != "GET":
54          abort(403)
55
56    post_number = int(request.args.get("q"))
57    q = "SELECT author, content FROM Post WHERE post_number='" + post_number +
"'"
58    post = dbquery(q).get_text()
59    q = "SELECT author, comment FROM Comments WHERE post_number='" +
post_number + "'"
60    comments = dbquery(q).get_text_list()
61
62    html_page = """< ... """ + post.author + """ posted ... """ + post.content
+ """ ... """
63    for i in range(len(comments)):
64        html_page += """ ... """ + comments[i].author """ replied ..."""
comments[i].comment
65    html_page += """...>"""
66
67    return html_page
68
```

The website uses a relational database as a back-end. The relevant database info can be inferred from the following sample table, as well as from the code:

User

username	password	admin
john	...	True
ikiga1	...	True
bebi	...	False
crimsonk	...	False

Comments

post_number	author	comment
237	anna	It is great indeed!
237	law	...
237	w4rio	...

Assume the following:
- The session is correctly handled (i.e., you can not forge arbitrary cookies).

- The function **`is_alphanum_plus()`** returns true only if the input string contains only letters, numbers, the whitespace, a comma, a full stop, and a question mark e.g., **`is_alphanum_plus`**("Hey, Jim. Shall we play 3 songs rather than 2?") returns true.
- The function **`html_escape`** correctly escapes all the html entities. I.e., all the html special characters are converted into their data equivalent (> becomes >).
- The function **`dbquery`** sends a query to the database and returns the response. To check if the response is empty one can call the method **`empty`** on the query response object.
- The library used to access the DBMS does not support stacked queries, i.e., the DBMS does not execute multiple queries if asked on the same query and separated by a semicolon. For instance, the query "select x from y where a=b; update k set [....] " would trigger an error and not be executed.

1. [3 points] Fill out the following table by answering the questions for each class of vulnerabilities. While answering the questions, be **exhaustive, concise, and straight to the point**. If you believe there might be a vulnerability, specify all the corresponding line/s of code.

Vulnerability class	*Is there a vulnerability of this class in the code above? How could an adversary exploit it?*	*If the vulnerability is present, explain the simplest procedure to remove it.*
Sql Injection (SQL-I)	*Yes, line 26, in the login function. There's no check on the username variable, which is directly inserted in a query. Line 46, the comment is html_escaped, but that does not prevent injections!*	*The simplest procedure is to fix line 23 and make it like line 08, checking is_alphanum_plus on the username too. Otherwise switch everything to prepared_statements etc.*
cross-site scripting (XSS) **Specify if reflected, stored, DOM...**	*Yes, because of line 26, 31, 46 and 64. That happens because of the SQLi: one could control the session object and bypass the username sanitization. If you correctly argue why not (see question 3), that could be acceptable.*	*The simplest procedure would be to apply escaping/filtering to the session's username. Fixing the SQLi would fix the issue, but it'd be better to fix line 46 and check if the user is alphanum. Otherwise, we could escape everything in get_post ...*
Cross site request forgery (CSRF)	*Yes, on the comment function which is a state-changing action [...]*	*See slides (CSRF-tokens, etc.).*

2. [2 points] Can you log in without having any registered users? Explain your exploit in detail and specify what would be your username once logged in.

We can leverage the SQL injection at line 26 and bypass the login procedure.
Send a post to https://bluepwnercult.com/login
username = ‘ OR 1=1; –
You will be logged in with username “‘ OR 1=1; –” (see the session management around line 31)

3. [2 points] While browsing the website, you came across this post from a new hacker in town.
https://bluepwnercult.com/post?p=237

bebi posted:
[...] It wasn't until the car suddenly stopped
In the middle of a cold and barren plain…

Love this song <3

Comment section
anna replied: It is great indeed!
law replied: wasn’t it a barren plateaus?
w4rio replied: @law please stop it with those quantum jokes…

You look at your browser and discover that this post’s number is 237. Describe how to carry out an exploit so that the browser of everyone that visits this page pops up a warning message that says “They're ok, the last days of may…”. Write the answer clearly, in detail, explaining every step.

The key is to use the SQL injection in the login function to set the session cookie with a username that contains a javascript payload. Then one can just comment post number 237 with any comment. Your username could contain unsanitized javascript.
The main problem is to come up with a polyglot username that:

- Works with the Select query at line 26
- Works with the Insert query at line 46
- Contains the string <script>alert(“They're ok, the last days of may…”)</script>

For instance, the username ‘ OR 1=1 OR password=’<script>alert(“They're ok, the last days of may…”)</script>’ satisfies the first and the last conditions, but breaks the Insert at line 46.

The student can either discuss the issue in this way or prove that there is no string that can satisfy both the first and the second requirement at the same time.

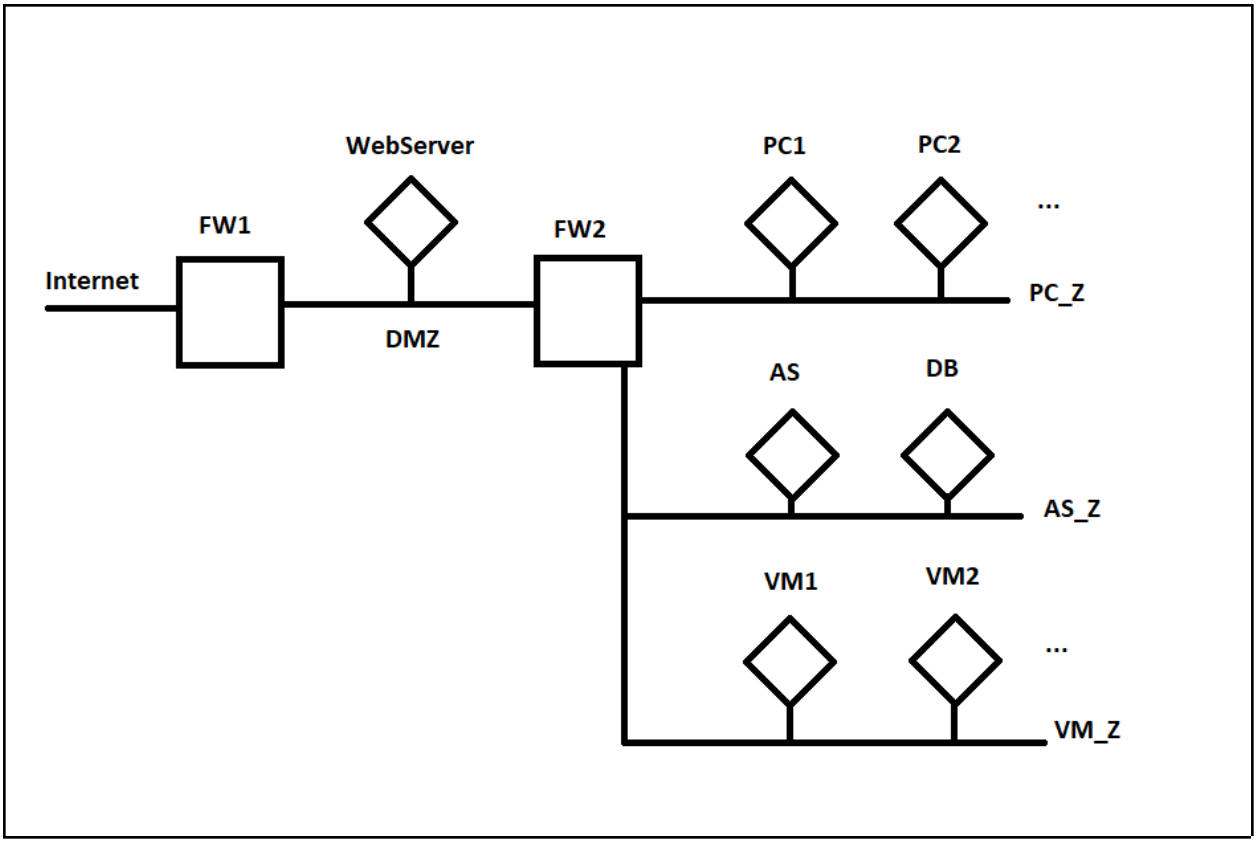
4. [1 point] The admins say that it is impossible for an attacker to log in as "ikiga1" without interacting with him. Prove them wrong.

We can use a blind SQL injection to recover ikiga1's password and then login as him.
Again, the injection is in login, on the username.
Username = "whatever' AND EXISTS (SELECT * FROM User WHERE username='ikiga1' AND password LIKE a%)"
If ikiga1's password starts with "a" the login will be successful. Otherwise the login will fail. Via bruteforce the attacker can learn ikiga1's password.

Exercise 3 (8 points)

The transport company that handles public transportation in a small city (buses, a small metro line) is setting up the network for their office, situated in the metro line premises. The office employees, from their desktop computers, need to access the **Internet** for work purposes (e.g., accessing their web-based email) as well as use an **internal web application,** served from the branch web server over the **HTTP** protocol. The web server also hosts the **online website**, served over the **HTTPS** protocol, from which customers can access information regarding the current status of the bus and metro lines, and buy tickets if they previously bought the physical card. The web server is backed by an **application server,** which stores its data on a **database server**. As the information processed by the application server and stored in the database server is sensitive, there is a strong requirement to prevent the employees from directly accessing those servers.
Besides the employee computers, the office has some **ticket vending machines** that allow the customers to self-service buy tickets and update travelcards. To process these transactions, vending machines communicate with the application server over a proprietary protocol and do not require Internet access.

1. [1 point] Please schematize the structure of the network, possibly aiding yourself with a graphical schema. Show how would you use firewalls to divide the various subnetworks, and which elements presented above would you put into each subnetwork. **Briefly justify** your choices.



2. [2 points] Write the firewall rules, assuming firewalls to be stateful packet filters (i.e. you can consider the response rules implicit).

Firewall	Src IP	Src PORT	Direction of the 1st packet	Dst IP	Dst PORT	Policy	Description
FW1 (example)	*10.0.0.1 (example)*	*ANY*	*zone 1 -> zone 2*	*192.168.0.2 (example)*	*443*	*DENY*	*(example: the X server in zone 1 cannot contact the Y server)*
FW1/2	ANY	ANY	ANY	ANY	ANY	DENY	Default Deny
FW1	ANY	ANY	Internet->DMZ	WS_IP	443	ALLOW	Online WS Access
FW1	Any IP in PC_Z	ANY	DMZ-Internet	ANY	ANY	ALLOW	Internet access for employees
FW2	Any IP in PC_Z	ANY	PC_Z → DMZ	ANY	ANY	ALLOW	Internet access for employees + internal webapp access
FW2	WS_IP	ANY	DMZ → AS_Z	AS_IP	AS_PORT	ALLOW	Web server → application server
FW2	Any IP in VM_Z	Prop.	VM_Z → AS_Z	AS_IP	Prop.	ALLOW	Vending machines → application server

3. [1 point] Let's now consider a more realistic scenario, where this office is only a small branch of a larger company that owns multiple offices around the city. The branch still require access for their employees, and the internal web application remains on-premises on the webserver, but the website, application server, and the database are now shared among various branches and kept in a central location, where they need to be made remotely accessible from each branch (and from the company branches only). How would you securely realize this architecture? Please state your assumption and detail any changes to the network diagram for this scenario.

The AS, DB and part of the WS are now in a remote location. As we don't want to expose them over the Internet, we need to set up a VPN between our network and the central branch. Basically we can accomplish this by setting up a VPN between the remote location and placing the VPN client in AS_Z to bridge the AS_Z network with the remote network (or assuming to set up a firewall-to-firewall VPN with the appropriate policies). As the overall network structure is unchanged, except for the VPN tunnel, the firewall policies would be the same.

4. [1 point] The employees of the branch have full access to the Internet. Thus, the company is worried that their computer could become infected with malware, and therefore decide to install a system to analyze the content of any HTTP response and scan it with anti-virus for the presence of known malware. Assume we're interested in filtering traffic to HTTP pages only. What kind of packet filter should the company put between the employees' LAN and the Internet zone? Why?

As we need to analyze the content, we need an application proxy (an HTTP proxy in particular).

5. [3 points] As a network expert, you realize that it would be possible to connect to a network outlet on the employee's network and *intercept all the HTTP communication between an employee and the branch web server exploiting the gateway.*

(a) Which technique could you use to reach this goal? State the name and detail all the steps that you should perform in order to intercept the communication between an employee's computer and the web server *in this specific scenario*.

We can use ARP spoofing to pose as the network gateway and sniff all the communication between the employee's computer and the gateway, including the traffic to the WS.
1. We first learn the IP address (e.g., by obtaining it via DHCP, or, if DHCP is not enabled, by passively sniffing the network broadcast traffic) and the real MAC address of the gateway (via ARP);
2. Then, we broadcasts ARP messages with the gateway IP address and our MAC address;
3. If the spoofing succeeds, the traffic to the gateway is directed to us (we can also forward traffic to the real gateway). In this way we are able to sniff all the information between the client and the gateway and, thus, between the client and the local web server.

(b) Can the same attack be used to intercept communication between the branch web server and the application server?

No, as they are on different networks.

Exercise 4 (9 points)

Part 1. Our systems have been compromised by very powerful malware. Luckily, we managed to collect a sample of the malware. Its code is reported below.

Sample

```
.text
0x08048040<evil>:
 0x08048040:    push   ebp
 0x08048041:    mov    esp, ebp
 0x08048043:    push   ebx

 0x08048044:    nop
 0x08048045:    add    eax, 0x10
 0x0804804b:    sub    eax, 0x10
 0x08048051:    lea    edx, 0x08060000
 0x08048057:    push   edx
 0x08048058:    call   0x080482e0 <system@plt>
 0x0804805d:    nop
 0x0804805e:    nop

 0x0804805f:    leave
 0x08048060:    ret


.rodata
0x08060000: "rm -rf /"
```

Strace

```
…
execve("/bin/rm", ["/bin/rm", "-rf", "/"], NULL) = 0
…
```

- **system** executes the command specified by the string parameter passed to it. For example, system("ls -l") executes "ls -l". Internally, it calls the "execve" function.
- **/bin/rm** removes all the files in the specified folder (in this case "/"). If **-rf** is specified, all the subdirectories are removed recursively.
- While the first box shows a snippet of the malicious code, the second box is its **strace**. **Strace** executes a program, capturing all the system calls that are performed.

[2 points] Does the malware show any evasive behaviors? If yes, specify which techniques are implemented.

No the malware is not showing any evasive behavior.

Part 2. We managed to recover a second sample from another machine. Its code is reported below.

Sample

```
.text
0x08048040<evil>:
 0x08048040:    push   ebp
 0x08048041:    mov    esp, ebp
 0x08048043:    push   ebx

 0x08048044:    mov    eax, 0x08070000
 0x08048049:    mov    ebx, 0x08048068
 0x0804804e:    mov    cl, BYTE PTR [eax]
 0x08048050:    mov    dl, BYTE PTR [ebx]
 0x08048052:    xor    dl, cl
 0x08048054:    mov    BYTE PTR [ebx], dl
 0x08048056:    add    eax, 0x1
 0x08048059:    add    ebx, 0x1
 0x0804805c:    mov    cl, BYTE PTR [eax]
 0x0804805f:    cmp    cl, 0x00
 0x08048062:    jne    0x0804804e

 0x08048068:    shr    ah, 0x86
 0x0804806b:    add    eax, 0xb78b86b1
 0x08048070:    (bad)
 0x08048071:    (bad)
 0x08048072:    leave
 0x08048073:    outs   dx, DWORD PTR ds:[esi]
 0x08048074:    xchg   cx, ax
 0x08048076:    test   BYTE PTR [edx-0x6ff8234b], dh
 0x0804807c:    hlt
```

```
 0x0804807d:    (bad)
 0x0804807e:    xchg   cx, ax
 0x08048080:    xchg   bx, dx
 0x08048082:    (bad)

 0x08048083:    leave
 0x08048084:    ret


.rodata
0x08060000: "rm -rf /"
0x08070000: <Random data with length 27, null terminated>
```

Strace

```
…
execve("/bin/rm", ["/bin/rm", "-rf", "/"], NULL) = 0
…
```

[2 points] The analysts are arguing whether this sample is the same as the one collected on the first machine. Do you also believe that this program is a variant of the first one that was encountered? If **yes**, specify what led you to think so. If **not**, explain what this new malware is doing.

It's the same as before, but the malicious code is encrypted. You can guess it's the same as above because the strace is the same and, in the end, it is also attempting to delete the entire filesystem.

[2 points] It is clear that the malware is showing evasive behavior. What technique is implemented?
In particular, if you believe that:

- This malware is metamorphic, then explain which instructions implement the obfuscation functionality;
- This malware is evasive, then explain how it is evading the analysis environment;
- This malware is polymorphic, then describe how the encryption/decryption routine is implemented.

It's polymorphic malware, because it is essentially decrypting the payload before executing it.

Moreover, there's no "mutation engine" in the code, so we can't say that it's metamorphic. On the contrary, the code in memory will always be the same. We don't have enough information to say whether it implements any evasion technique.

The encryption algorithm is a one time pad with one single key that is as long as the encrypted payload.

Part 3. A third sample is collected from yet another machine. Its disassembly and strace are reported in the boxes below.

Sample

```
.text
0x08048040<evil>:
 0x08048040:    push   ebp
 0x08048041:    mov    esp, ebp
 0x08048043:    push   ebx

 0x08048044:    mov    eax, 0x08070000
 0x08048049:    mov    ebx, 0x08048068
 0x0804804e:    mov    cl, BYTE PTR [eax]
 0x08048050:    mov    dl, BYTE PTR [ebx]
 0x08048052:    xor    dl, cl
 0x08048054:    mov    BYTE PTR [ebx], dl
 0x08048056:    add    eax, 0x1
 0x08048059:    add    ebx, 0x1
 0x0804805c:    mov    cl, BYTE PTR [eax]
 0x0804805f:    cmp    cl, 0x00
 0x08048062:    jne    0x0804804e

 0x08048068:    shr    ah,0x86
 0x0804806b:    add    eax,0xb78b86b1
 0x08048070:    (bad)
 0x08048071:    (bad)
 0x08048072:    leave
 0x08048073:    outs   dx, DWORD PTR ds:[esi]
 0x08048074:    xchg   cx, ax
 0x08048076:    test   BYTE PTR [edx-0x6ff8234b], dh
```

```
 0x0804807c:    hlt
 0x0804807d:    (bad)
 0x0804807e:    jmp    0xfe
 0x08048083:    xchg   di, si
 0x08048085:    in     ax, dx
 0x08048087:    (bad)
 0x08048088:    xchg   ax, bx
 0x0804808a:    (bad)

 0x0804808b:    leave
 0x0804808c:    ret


.rodata
0x08060000: <Random data with length 8, null terminated>
0x08070000: <Random data with length 35, null terminated>
```

Strace

```
…
sleep(0x1000)
…
```

[2 points] This third malware seems to be heavily obfuscated as well. Hence, basic static analysis does not produce much information. For this reason, the analysts tried to execute it in a sandbox for a few minutes. Surprisingly, nothing bad happened; on the contrary, the filesystem of the sandbox looked intact. Can we conclude that this sample is different, in terms of functionalities, from the ones seen before? If **yes**, explain what this new malware is doing. If **not**, explain why the filesystem was not modified by the malware.

We don't have enough information to conclude that this sample is the same or a different one, although we can see that the decryption routine is the same as the one in part 2.

What we can say is that there's a long sleep in the strace output. Therefore, it's likely that the analysts did not run the binary for long enough. Thus, the malware had not shown its true behavior yet when the analysis was interrupted .

[1 point] How would you change the approach taken by the analysts to obtain more information on this last sample?

Since the strace shows a call to the sleep function with 0x1000 as a parameter, then we have to wait for at least 0x1000 seconds in order to see the true behavior of the malware.